MSC DISSERTATION PROTOTYPE

DeepGuard

A multimodal deepfake detection system — fusing visual, audio and lip-sync evidence into one interpretable verdict.

DOCUMENT

Complete project explanation — methodology, technology, architecture and flow

SYSTEM TYPE

Full-stack web application (FastAPI backend · Next.js frontend)

DETECTION BRANCHES

EfficientNet-B0 (visual) · Wav2Vec2 + MLP (audio) · SyncNet-style (lip-sync) · Fusion head

REPOSITORY LAYOUT

backend/ — FastAPI · SQLAlchemy · SQLite | frontend/ — Next.js 14 · TypeScript · Tailwind

> **Status disclosure.** The detection models are specified and the system is complete, but the models are not yet trained. The application currently runs `MockPipeline`, which produces synthetic scores. Figures shown in the interface must not be cited as experimental results.

Contents

1. The one-paragraph version
   What DeepGuard is, in a single statement
2. The problem, and why multimodal
   Research justification and motivation
3. Methodology
   The four models, the five-stage pipeline, score semantics, explainability
4. System architecture
   Three layers and the Strategy pattern that separates them
5. Technology stack
   Every dependency and the reason it was chosen
6. End-to-end flow
   One video traced through the entire system, step by step
7. Design decisions to defend
   Trade-offs and their justifications
8. UI/UX engineering
   Theming, accessibility and interaction design
9. Limitations
   Honest scope statement
10. Anticipated questions
    Viva preparation with model answers

1 The one-paragraph version

> DeepGuard is a full-stack web application for multimodal deepfake detection. Instead of judging a video on appearance alone, it scores the clip along three independent axes — visual artefacts, audio synthesis markers, and lip-sync alignment — and fuses those into a single verdict with a confidence value. The system is built as a FastAPI backend with a Next.js frontend, and is architected so the detection models can be swapped in behind a fixed interface without changing a single route or UI component.

2 The problem, and why multimodal

This is the research justification — typically the first thing an examiner will probe.

2.1 Single-modality detectors generalise poorly

DETECTOR TYPE	HOW IT FAILS
Visual only	Defeated by heavy compression — social-media re-encoding destroys the pixel-level artefacts it depends on.
Audio only	Says nothing at all about a face swap laid over genuine, unmodified speech.
Lip-sync only	A well-made deepfake with matched audio can preserve alignment.

2.2 The insight

These three failure modes are largely **independent**. A forgery that defeats one detector usually leaves evidence for another. Face-swap and voice-cloning pipelines in particular break the audio-visual coupling even when each stream is individually convincing.

2.3 The second contribution: interpretability

Because each branch reports its own score *and* a written rationale, a result is auditable. You can see whether the evidence came from pixels, from the waveform, or from the mismatch between them. A black box that says "87% fake" is useless in a forensic context; DeepGuard shows its working.

3 Methodology

3.1 The four models

BRANCH	MODEL	WHAT IT DETECTS
Visual	EfficientNet-B0, fine-tuned on face crops	Blending seams, inconsistent skin texture, frequency artefacts from generative models
Audio	Wav2Vec2 encoder + lightweight MLP head	Spectral envelope, formant transitions, vocoder periodicity, missing breath noise
Lip-sync	SyncNet-style two-stream network	Audio-visual offset between mouth-region video and the speech envelope
Fusion	Fusion classifier over the three scores	Learns how much to trust each modality given their agreement pattern

3.2 The processing pipeline — five stages

Defined canonically in `backend/app/inference/base.py`, each stage owning a progress band:

```
extracting_frames    0 → 20 %    decode video, MTCNN face detection, crop, save thumbnail
visual_analysis     20 → 45 %    EfficientNet-B0 per frame + Grad-CAM overlays
audio_analysis      45 → 65 %    Wav2Vec2 embeddings → MLP
lipsync_analysis    65 → 85 %    two-stream AV alignment scoring
fusion              85 → 100 %   combine branch logits → verdict + confidence
```

This single tuple is the contract: the backend reports these stage keys, the frontend renders them. Add a stage in one place and the progress UI follows automatically.

3.3 Score semantics

> **GET THIS RIGHT IN THE VIVA**
>
> Two different numbers that are easy to confuse:
>
> - **Modality score = P(manipulated)** for that branch alone, 0–1. Higher means stronger manipulation evidence.
> - **Confidence = certainty in the reported verdict** — *not* the probability of being fake.

The fusion in the current implementation is a weighted sum:

```
fused      = 0.45 · visual + 0.28 · audio + 0.27 · lipsync
verdict    = "fake" if fused ≥ 0.5 else "real"
confidence = fused if fake else (1 − fused)
```

That last line is the important one: a clip scoring `0.05` fused gets verdict `real` with **95% confidence** — not 5%.

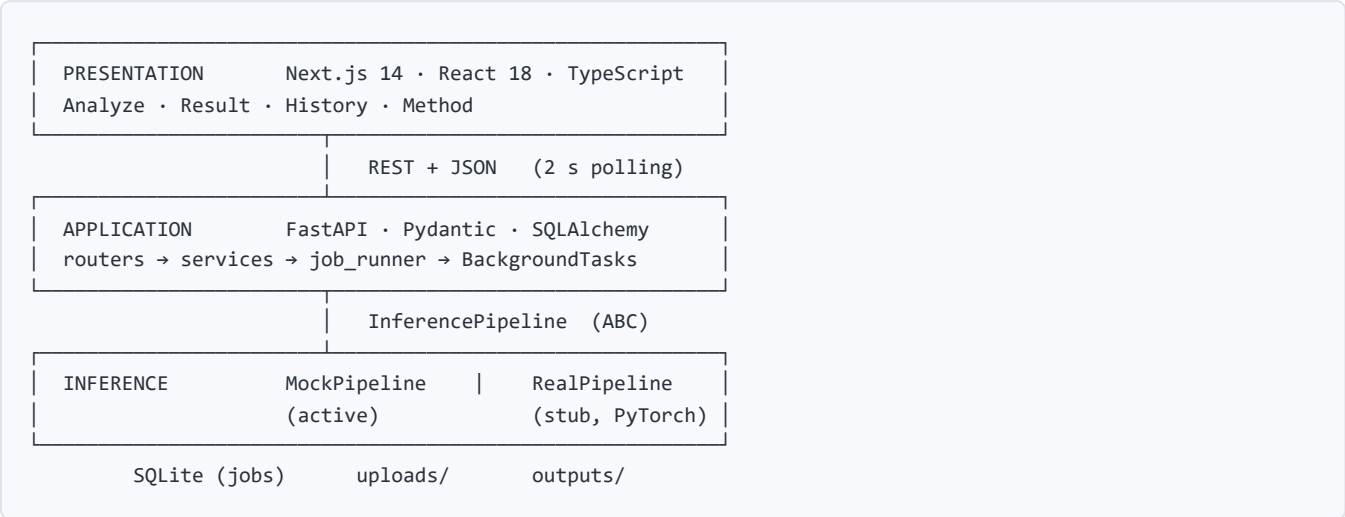
3.4 Explainability

Grad-CAM (Gradient-weighted Class Activation Mapping) is applied to the visual branch to produce heatmap overlays showing which face regions drove each frame-level decision. These are written to disk per job and displayed as a clickable strip on the result page.

Disagreement is deliberately surfaced, not hidden. A clip where only the lip-sync branch objects tells a very different story from one where all three agree. The mock pipeline even injects a dissenting branch 30% of the time so the interface is tested against that case.

4 System architecture

Three cleanly separated layers:



4.1 The key architectural decision: the Strategy pattern

`InferencePipeline` is an abstract base class with one abstract method:

```
def analyze(self, video_path, progress_callback, output_dir) -> ResultDict
```

The entire API layer talks *only* to this interface. `get_pipeline()` reads an environment variable and returns either implementation, cached with `@lru_cache` so models load once per process. `RealPipeline` is imported **lazily** — inside the function, not at module top — so the mock stack never pays the cost of importing PyTorch.

> **WHY THIS MATTERS FOR YOUR DEFENCE**
>
> It decouples the *research* contribution (the models) from the *engineering* contribution (the system). The complete working system could be demonstrated before a single model was trained, and swapping in real models requires zero changes to routes, schemas, or frontend code.

5 Technology stack — and why each choice

5.1 Backend

TECHNOLOGY	VERSION	WHY THIS ONE
FastAPI	0.115.6	Async-native, auto-generates OpenAPI docs at `/docs`, and `BackgroundTasks` gave long-running inference without a separate queue
Pydantic	2.10.4	Every response is a validated model; constraints declared inline (`score: float = Field(ge=0.0, le=1.0)`) so invalid data cannot reach the client
SQLAlchemy	2.0.36	Typed ORM using modern `Mapped[]` syntax; DB-agnostic, so SQLite→PostgreSQL is a URL change
SQLite	bundled	Zero-configuration, single file — right for a single-user prototype
Uvicorn	0.34.0	ASGI server
Pillow	11.1.0	Thumbnail and heatmap rendering — keeps the mock stack free of OpenCV/torch
pydantic-settings	2.7.0	All configuration from `DEEPGUARD_*` environment variables; nothing hardcoded
python-multipart	0.0.20	Multipart parsing for the video upload endpoint

5.2 Frontend

TECHNOLOGY	VERSION	WHY THIS ONE
Next.js	14.2.23	App Router, file-based routing, production build optimisation
React	18.3.1	Component model; hooks for polling state
TypeScript	5.7.3	Strict mode; `lib/types.ts` mirrors the Pydantic models so API drift becomes a compile error
Tailwind CSS	3.4.17	Utility-first; the whole theme is CSS variables, so light/dark needs no duplicated classes
jsPDF	2.5.2	Client-side PDF reports — no server round-trip

5.3 For the real pipeline (currently commented out)

LIBRARY	VERSION	ROLE
torch	2.5.1	Inference and training runtime for all four networks
torchvision	0.20.1	EfficientNet-B0 backbone and transform pipeline
transformers	4.47.1	Pre-trained Wav2Vec2 encoder
torchaudio	2.5.1	Waveform loading, resampling, mel-spectrograms
facenet-pytorch	2.6.0	MTCNN face detection — gates the visual and lip-sync branches
opencv-python	4.10.0.84	Video decoding, mouth-region cropping, Grad-CAM compositing

System-level: FFmpeg (non-optional — `.avi` / `.mov` decoding and audio demuxing), CUDA 12.1/12.4 + cuDNN 9, NVIDIA driver ≥ 550.

6 End-to-end flow

This is the heart of the demonstration. Follow one video through the entire system.

1. **Upload (browser)**

   The user drags an MP4 onto the drop zone. The client validates **before** sending: extension against the allow-list, size against the 100 MB cap, and rejects empty files. An object URL provides an instant preview.

2. **POST /api/analyze**

   Sent via **XMLHttpRequest, not fetch** — a deliberate choice, because XHR exposes `upload.onprogress` and fetch does not, so the user sees a real percentage for large files.

3. **Server-side validation and storage**

   `validate_upload()` re-checks extension *and* declared MIME type — never trust the client. Then `save_upload()` streams the file to disk in **1 MiB chunks**, tracking cumulative size and aborting mid-write past the limit:

   ```
   while chunk := await file.read(CHUNK_SIZE):
       size += len(chunk)
       if size > settings.max_upload_bytes:
           destination.unlink(missing_ok=True)
           raise HTTPException(413, ...)
   ```

   Why chunked? A naive `await file.read()` loads the whole 100 MB into RAM. This caps memory usage regardless of file size — and rejects an oversized file after ~1 MB instead of after the whole upload completes.

   The file is stored as `{uuid4}.mp4`. UUID naming prevents collisions and path-traversal via crafted filenames, while the original name is kept separately in the database for display.

4. **Job row and background task**

   A `Job` row is inserted with status `queued`, then:

   ```
   background_tasks.add_task(run_analysis_job, job_id)
   return JobCreated(...)      # HTTP 202 Accepted
   ```

   202, not 200 — the correct code for "accepted, processing not complete". The response returns immediately; the browser is never left hanging.

5 Background inference

`run_analysis_job` runs in FastAPI's threadpool. It:

- Opens **its own database session** — it is on a different thread from the request
- Sets status → `processing`
- Defines a `progress_callback` closure writing stage and percent straight to the `jobs` table
- Calls `pipeline.analyze(video_path, progress_callback, output_dir)`

Progress writes are throttled (epsilon = 0.9): the database is only written when the bar moves about 1%, preventing hundreds of writes per run. The whole body is wrapped in `except Exception` — a crashing job marks itself `failed` with the error message and **never kills the worker**.

6 Polling

Meanwhile the frontend runs `useJobPolling` — `GET /api/jobs/{id}` every 2 seconds. Critically it uses **recursive `setTimeout`, not `setInterval`**, so requests cannot stack up if one is slow, and it **stops automatically** when status becomes `done` or `failed`. On network error it backs off to 4 seconds and keeps retrying.

7 Persisting the result

On completion the full `ResultDict` is stored verbatim as JSON in `result_json`, *and* key fields are copied into dedicated columns (`verdict`, `confidence`, `visual_score`, …).

That is deliberate denormalisation. The history grid needs to sort and display nine jobs at once — reading indexed columns is far cheaper than parsing nine JSON blobs. The JSON retains full fidelity for the detail view.

8 Serving the result

`build_result()` converts stored *relative* paths into *absolute* URLs using `settings.public_base_url`. Two static mounts serve them: `/static` for generated artefacts and `/media` for uploaded videos.

Why store relative and build absolute at response time? Change the deployment domain and every historical record's URLs update automatically — no database migration required.

9 Result page

The frontend redirects to `/result/{jobId}`, showing the verdict banner, an animated confidence gauge, three modality cards, the Grad-CAM strip with lightbox, the source video player, and run metadata. jsPDF builds a downloadable report client-side.

6.1 API surface

METHOD	PATH	DESCRIPTION
POST	`/api/analyze`	Multipart upload; validates, stores, creates job, launches inference. Returns `202`.
GET	`/api/jobs/{id}`	Status, stage, progress 0–100, and on completion the full result. Polled every 2 s.
DELETE	`/api/jobs/{id}`	Deletes the job row, the upload and all generated artefacts.
GET	`/api/history`	Paginated past jobs with thumbnails and verdicts.
GET	`/health`	Liveness, whether models are loaded, jobs in flight.
GET	`/static/…`	Generated Grad-CAM frames and thumbnails.
GET	`/media/…`	Uploaded videos, so the result page can replay the source clip.

7 Design decisions to defend

DECISION	JUSTIFICATION
BackgroundTasks, not Celery/Redis	Inference takes about 15 seconds and this is a single-user prototype. Celery would add a broker, a worker process and deployment complexity for no benefit at this scale. The limitation is known and the migration path can be stated.
Polling, not WebSockets	Polling is stateless, survives reconnects, needs no extra protocol, and 2-second granularity is imperceptible for a 15-second job. WebSockets would add connection lifecycle management for negligible gain.
SQLite, not PostgreSQL	Zero-configuration, single file, appropriate for one user. `check_same_thread=False` is set because the background worker touches the session from another thread.
Mock pipeline built first	Let the entire UI, API contract and error handling be built and tested before models existed — and lets the system be demonstrated even if training does not finish. Classic dependency inversion.
Environment-driven configuration	Nothing hardcoded. Every path, URL and limit lives in `config.py` behind the `DEEPGUARD_` prefix.
TypeScript mirroring Pydantic	Every API response has a Pydantic model server-side and a matching interface client-side. `npm run typecheck` catches contract drift at compile time.

8 UI/UX engineering

Worth presenting as an engineering contribution in its own right:

- **Light / dark / system theming** via CSS custom properties wrapped as `rgb(var(--token) / <alpha-value>)`, applied by a pre-paint inline script so there is no flash of the wrong theme on load.
- **Accessibility**: skip-to-content link, one consistent focus ring, `prefers-reduced-motion` support, `role="progressbar"` and `role="meter"` with ARIA values, focus-trapped modal dialogs.
- **Destructive-action safety**: deletion was previously a single irreversible click; it now requires explicit confirmation, since it removes the job row, the upload *and* all generated artefacts.
- **Responsive navigation** with a mobile drawer.
- **ETA estimation** on the progress view, derived from the observed completion rate.
- **Consolidated icon system** replacing roughly thirty inline SVGs that had inconsistent stroke weights.

9 Limitations

> **STATE THESE BEFORE YOU ARE ASKED**
>
> Owning your limitations is what separates a good viva from a bad one.

1. **The models are not implemented.** `RealPipeline` is a stub raising `NotImplementedError`. The system runs `MockPipeline`, whose scores come from a seeded random number generator. `model_version` reports `"mock-0.1.0"` and the Method page carries a disclaimer. These scores must not be cited as experimental results.
2. **Single-node by construction.** BackgroundTasks + SQLite + local disk means the GPU serialises across concurrent uploads. Multi-user deployment needs Celery and Redis, PostgreSQL, and S3 or a shared volume.
3. **No retention policy.** Artefacts are reclaimed only on explicit delete, so storage grows without bound.
4. **No authentication.** Any client that can reach the API can upload and delete.
5. **Scores are indicative, not forensic proof.** Performance degrades on heavily compressed or very short clips.
6. **Orphaned jobs.** If the backend crashes mid-job the row stays `processing` forever. A startup reconciliation step would fix this.

10 Anticipated questions

Q Why not just use one strong visual model?

> Because a face swap over genuine speech, and a voice clone over genuine video, are each invisible to a visual-only detector in one direction or the other. The modalities fail independently, which is precisely what makes fusing them worthwhile.

Q How does the fusion actually work?

> Currently a weighted linear combination (0.45 / 0.28 / 0.27, visual-weighted because it is the strongest individual signal). The intended design is a learned classifier that weights each modality according to their agreement pattern.

Q What happens if the backend crashes mid-job?

> The job stays `processing` in the database. That is a known gap — a startup reconciliation step marking orphaned `processing` jobs as `failed` would resolve it.

Q Why build a mock first? Is that not avoiding the hard part?

> It is dependency inversion. The interface is the contract; both the UI and the models are implementations against it. It allowed the entire system design — error handling, progress reporting, storage lifecycle — to be validated independently of model training, and it de-risks the project: the system demonstrates regardless of training outcomes.

Q How would you validate the real models?

> Train on FaceForensics++, Celeb-DF v2 and DFDC; report AUC, EER and accuracy per branch and for the fusion head; then test *cross-dataset* (train on one, test on another) to measure generalisation — which is exactly where single-modality detectors visibly break down.

Q Why polling rather than pushing updates?

> The job is short and the client is a single browser tab. Polling requires no connection state on the server, recovers automatically from a dropped network, and the implementation stops itself on a terminal status. WebSockets would introduce reconnection and heartbeat handling for a two-second improvement in latency that no user would perceive.

Q What is the difference between the score and the confidence?

> The per-modality score is P(manipulated) for that branch alone. The confidence is how certain the fusion classifier is about the verdict it reported. A clip judged authentic with a fused score of 0.05 is reported as *real* at 95% confidence — the two numbers are complements, not the same quantity.

Q What would you do differently with more time?

> Train and integrate the four networks; add cross-dataset evaluation; replace BackgroundTasks with a real queue so concurrent analysis is possible; add authentication and a retention policy; and add temporal modelling across frames rather than treating each frame independently in the visual branch.

PRESENTATION ADVICE

Do not let the polish of the interface imply that the detection works. The system is complete and the models are specified; the models are not yet trained. Say that plainly and early — an examiner who discovers it themselves will trust everything else you said less.

DeepGuard — MSc dissertation prototype. Research artefact; results are indicative, not forensic proof. Videos are processed by a locally hosted backend and are not transmitted to any third-party service.